Lab 6:

Set Up Nexus Repository and Configure User Access

Name	Mehmoona Abid
Roll number	2023-BCS-097
Section	A
GitHub repository link	https://github.com/mehmoonaabid/DevOps

Getting Started

1. Create the Lab 6 evidence directory in your existing DevOps repository:
2. cd ~/DevOps

mkdir -p Labs/Lab06/screenshots

3. Connect to the Ubuntu server using your registered GitHub username, unless you are already signed in locally:

ssh <github-username>@<ubuntu-server-ip>

4. Verify Docker directly on the Ubuntu server:

5. docker --version

docker ps

6. Keep the Lab 5 IntelliJ IDEA environment and these projects for Tasks 3 and 4:
7. /config/IdeaProjects/java-app
8. /config/IdeaProjects/java-maven-app

If either project does not exist, its task explains how to clone it. Do **not** use the IntelliJ container shell to run Nexus. Run every command in Tasks 1, 2, 5 and 6 from the Ubuntu server terminal.

Task 1: Run Nexus Repository and Complete the Initial Setup

1. **Start the Nexus container**

In the Ubuntu server terminal, run:

```
docker run -d \
  --name nexus \
  --restart unless-stopped \
  -p 8006:8081 \
  -p 8007:8007 \
  -v nexus-data:/nexus-data \
  sonatype/nexus3:latest
```

This command creates a container named nexus. Host port 8006 opens the Nexus web interface, host port 8007 is reserved for a later repository task, and the named volume nexus-data keeps Nexus data when the container restarts.

If you previously created the container and it is stopped, do not run the creation command again. Start the existing container with:

docker start nexus

2. **Verify that Nexus is running**

Run:

docker ps | grep nexus

Wait until the output shows the nexus container. The first startup can take several minutes. If the browser is not ready yet, inspect startup progress with:

docker logs -f nexus

Press **Ctrl+C** after Nexus finishes starting. This stops only the log display; it does not stop the container.

- Screenshot as nexus_container_running.png, showing docker ps | grep nexus, the running nexus container, and the published ports.

3. **Open the Nexus login page**

On a computer that can reach the Ubuntu Docker host, open:

http://<ubuntu-server-ip>:8006

Replace <ubuntu-server-ip> with the real IP address. The first login username is:

admin

Instructor reference image:

4. **Retrieve the one-time administrator password**

Return to the Ubuntu server terminal and run:

docker exec nexus cat /nexus-data/admin.password

Copy the generated value privately and use it once to sign in as admin. **Do not take a screenshot of this command or its output.** Clear the visible terminal immediately afterward:

clear

5. **Complete the setup wizard**

Complete the pages shown by Nexus in this order:

i. Select **Next** on **Let's Get You Set Up**.

ii. Enter and confirm a new strong password for the admin user, then select **Next**.

iii. Continue with **Nexus Repository Community Edition**.

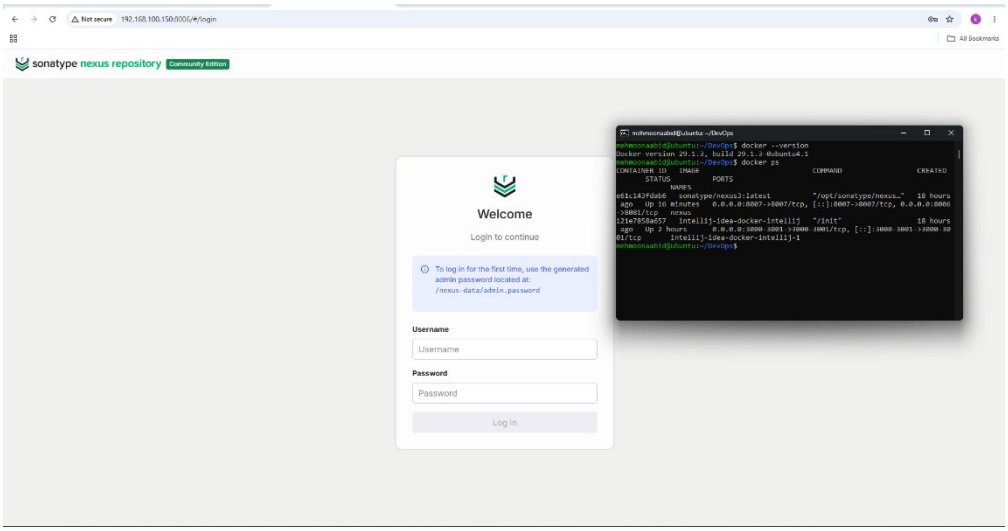
iv. Read the End User License Agreement, scroll as required, and select **Agree**.

v. On **Configure Anonymous Access**, select **Disable anonymous access**, then select **Next**.

vi. On **Setup Complete**, select **Finish**.

Screenshot Required:

- nexus_container_running.png



Task 2: Create a Role and User and Verify Access

1. **Create the nx-java role**

Sign in as admin, open **Settings**, and go to **Security → Roles**. Select **Create Role** and enter:

Field	Value
Type	Nexus role
Role ID	nx-java
Role Name	nx-java
Role Description	Maven snapshots role

Under **Applied Privileges**, select **Modify Applied Privileges**. Search for maven2-maven-snapshots, select this privilege, and confirm the selection:

nx-repository-view-maven2-maven-snapshots-*

After confirming the selection, verify that the privilege appears under **Applied Privileges**:

Save the role. Reopen the saved role if necessary to show its details.

2. **Create the local nexus user**

Go to **Settings → Security → Users**, select **Create local user**, and enter:

Field	Value
ID	nexus
First name	nexus
Last name	dev
Email	nexus@example.com
Password	A new strong password chosen by you
Status	Active
Granted role	nx-java

Move nx-java from **Available** to **Granted**, then select **Create local user**. Do not reuse the administrator password.

3. **Verify the user's access**

In the Ubuntu server terminal, run the following command after replacing <ubuntu-server-ip> with the real IP address:

curl -i -u nexus http://<ubuntu-server-ip>:8006/service/rest/v1/status

When curl displays Enter host password for user 'nexus':, enter the password created in the previous step. The password is not displayed. Do not place the password after nexus: in the command because that would save it in shell history and may expose it in the screenshot.

A successful request returns an HTTP status line containing:

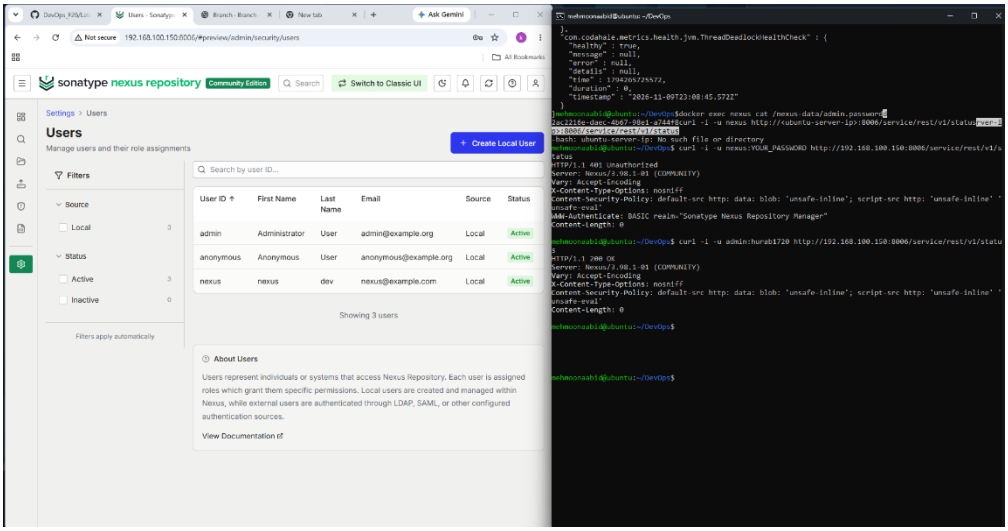
HTTP/1.1 200 OK

- Screenshot as nexus_user_access.png, showing the curl command, the HTTP/1.1 200 response, and your required terminal prompt. The password must not be visible.

If the response is 401 Unauthorized, verify the user ID, password, and Active status. If repository access later returns 403 Forbidden, verify that nx-java is granted to the user and contains the required Maven snapshots privilege.

Screenshot Required:

- nexus_user_access.png



Task 3: Build and Publish the Gradle Application to Nexus

Run the Nexus container on the Ubuntu server as configured in Tasks 1 and 2. Perform the project editing, Gradle build and Gradle publication commands in the **IntelliJ IDEA terminal** used in Lab 5.

1. **Clone or reopen the Gradle project**

From the IntelliJ Welcome screen, select **Clone Repository** or **Get from Version Control** and use:

- **URL:** https://gitlab.com/nanuchi/java-app.git
- **Directory:** /config/IdeaProjects/java-app

If /config/IdeaProjects/java-app already exists from Lab 5, do not create a duplicate clone. Select **Open**

or **Recent Projects** and open that existing directory.

2. **Configure the project JDK**

In IntelliJ IDEA, select **File → Project Structure**.

Under **Project Settings → Project**, configure:

Setting	Required value
Project SDK	JDK 16
Project language level	8 - Lambdas, type annotations, etc.

If JDK 16 is not listed, open the SDK menu and select **Download JDK**. You may use Azul Zulu or another JDK vendor, but select the architecture that matches your computer. The instructor image uses ARM64 because it was captured on an ARM server; students using x86_64 must select the matching x86_64 JDK.

Select **Apply**, then **OK**.

3. **Configure build.gradle for Maven publication**

Open build.gradle. Ensure its plugins block contains each plugin exactly once:

```
plugins {
    id 'java'
```

```
    id 'maven-publish'
    id 'org.springframework.boot' version
'2.2.2.RELEASE'
    id 'io.spring.dependency-management' version
'1.0.8.RELEASE'
}
```

Do not add a second java or maven-publish entry if it already exists.

Keep the project's existing Maven coordinates. Verify that the file contains:

```
group = 'com.example'
version = '1.0-SNAPSHOT'
```

Add the following publishing block immediately after the existing group and version lines. Replace <ubuntu-server-ip> with the IP address of the Ubuntu server running Nexus; do not include angle brackets in the final URL.

```
publishing {
    publications {
        maven(MavenPublication) {
            artifact("build/libs/my-app-$version" +
".jar") {
                extension "jar"
```

```
        }
    }
}
repositories {
    maven {
        name "nexus"
        url "http://<ubuntu-server-
ip>:8006/repository/maven-snapshots/"
        allowInsecureProtocol = true
        credentials {
            username project.repoUser
            password project.repoPassword
        }
    }
}
}
```

allowInsecureProtocol = true is required here because this classroom Nexus URL uses HTTP. Do not use this setting as a substitute for HTTPS in a production deployment.

4. **Set the project name**

Open settings.gradle and set its contents to:

rootProject.name = 'my-app'

This name makes the generated JAR begin with my-app.

5. **Create gradle.properties**

In IntelliJ's **Project** pane, right-click the java-app project root and select **New → File**.

Enter this filename and press **Enter**:

gradle.properties

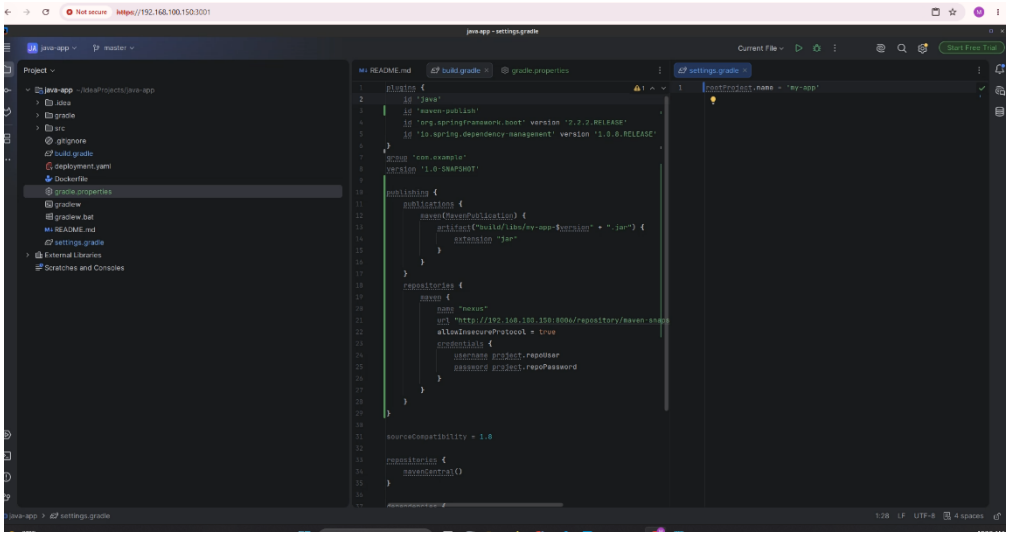
Add these two lines, using the nexus user's password created in Task 2:

repoUser=nexus

repoPassword=<nexus-user-password>

Save the project-level gradle.properties file. Never display its contents in a screenshot, commit it toGit, include it in Lab6_Solution.pdf, or expose its password in terminal output.

- Screenshot as gradle_nexus_configuration.png, showing the updated build.gradle publication/repository configuration and settings.gradle. Keep the required terminal identity visible, but do not show gradle.properties or either password.



6. **Build the JAR**

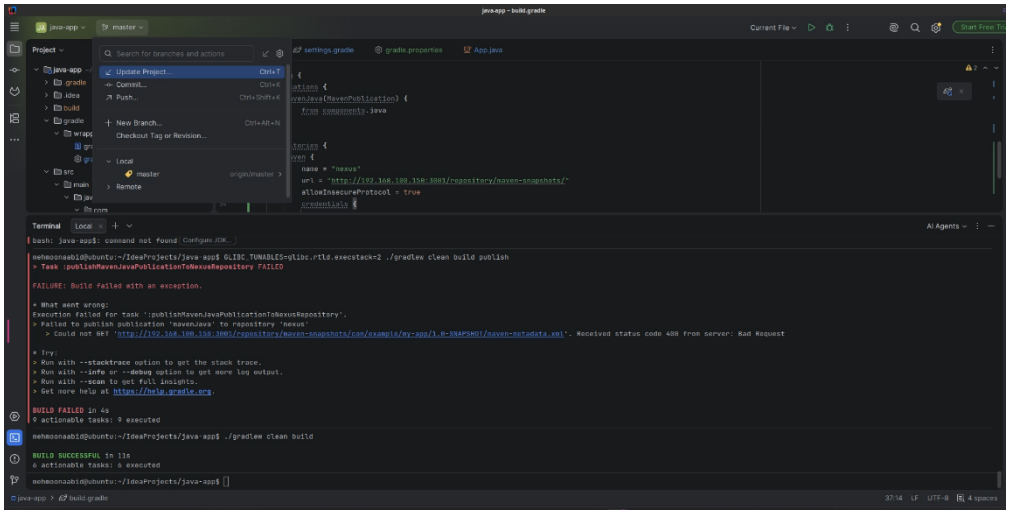
Open IntelliJ's terminal in the java-app project and run:

./gradlew clean build

Wait for BUILD SUCCESSFUL. In IntelliJ's Project pane, expand build/libs/ and confirm that it contains:

my-app-1.0-SNAPSHOT.jar

- Screenshot as gradle_nexus_build.png, showing ./gradlew clean build, BUILD SUCCESSFUL, and the generated my-app-1.0-SNAPSHOT.jar.



7. **Publish the JAR to Nexus**

Run:

./gradlew clean build publish

Gradle should run the Nexus publication task and finish with BUILD SUCCESSFUL.

- Screenshot as gradle_nexus_publish.png, showing the command, the Nexus publish task and BUILD SUCCESSFUL. Do not display the credentials file or password.



If publication returns 401 Unauthorized, verify repoUser, repoPassword and the Active status of the

nexus user. If it returns 403 Forbidden, confirm that the user has the nx-java role and that the role contains nx-repository-view-maven2-maven-snapshots-*.

8. **Verify the artifact in Nexus**

Open the Nexus web application:

http://<ubuntu-server-ip>:8006

Sign in and open the navigation menu if it is collapsed.

Select **Browse**, then select the hosted maven-snapshots repository.

Expand this path:

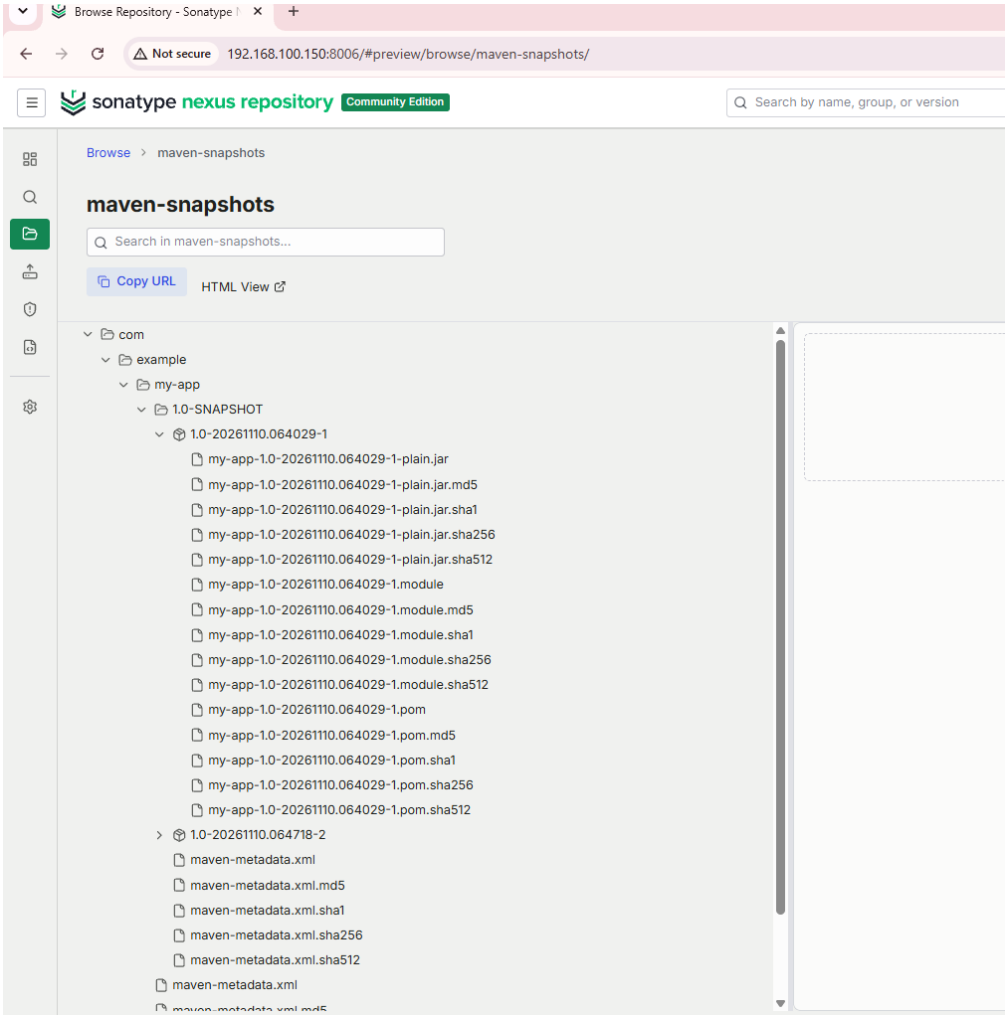
com → example → my-app → 1.0-SNAPSHOT

Expand the 1.0-SNAPSHOT folder and confirm that Nexus contains the published JAR, POM and Maven metadata/checksum files. Nexus may store the snapshot JAR with a timestamp and build number rather than the literal SNAPSHOT filename.

The instructor reference image shows the repository hierarchy and metadata. For your required evidence, expand 1.0-SNAPSHOT one level further so the JAR filename is also visible.

- Screenshot as nexus_gradle_artifact.png, showing maven-snapshots, the expanded

com/example/my-app/1.0-SNAPSHOT path, the published JAR and the metadata. Keep the required terminal prompt visible beside the browser.



Task 4: Publish the Maven Application to Nexus

Keep the Nexus container running on the Ubuntu server. Perform the Maven project editing,

packaging and deployment commands in the **IntelliJ IDEA terminal** used in Lab 5.

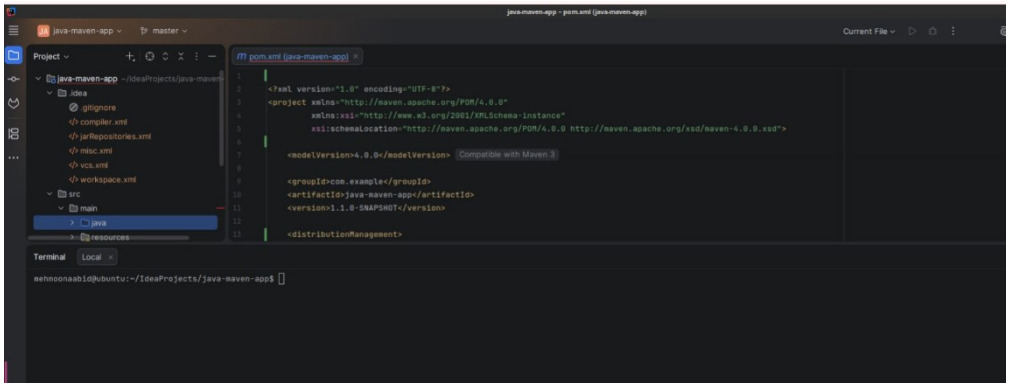
1. **Clone or reopen the Maven project**

From IntelliJ's Welcome screen, select **Clone Repository** or **Get from Version Control** and use:

- **URL:** https://gitlab.com/nanuchi/java-maven-app.git
- **Directory:** /config/IdeaProjects/java-maven-app

If /config/IdeaProjects/java-maven-app already exists from Lab 5, do not clone it again. Select **Open** or **Recent Projects** and open the existing directory.

- Screenshot as maven_nexus_project.png, showing the successfully opened java-maven-app project in IntelliJ.



2. **Add snapshot distribution management to pom.xml**

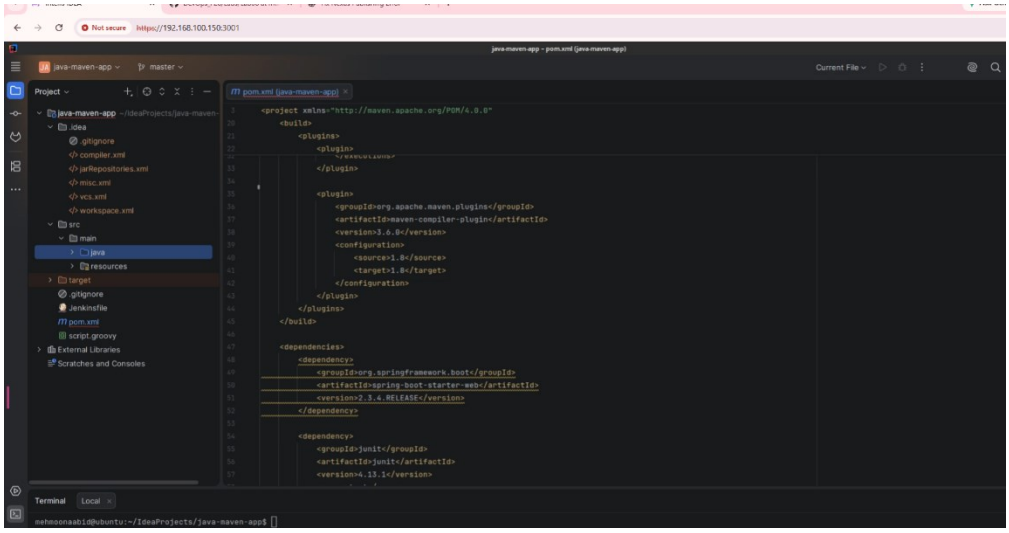
Open pom.xml. Add this block as a direct child of the top-level <project> element, immediately after the project's existing <version> line. Do not place it inside <dependencies> or <build>.

Replace <ubuntu-server-ip> with the IP address of the Ubuntu server running Nexus. Keep the URL on one line.

```
<distributionManagement>
  <snapshotRepository>
    <id>nexus-snapshots</id>
    <url>http://<ubuntu-server-ip>:8006/repository/maven-snapshots/</url>
  </snapshotRepository>
</distributionManagement>
```

The project version ends with SNAPSHOT, so Maven uses this snapshotRepository when it deploys the artifact.

- Screenshot as maven_distribution_management.png, showing the complete saved distributionManagement block and the surrounding top-level pom.xml structure. Keep the required terminal identity visible.



3. **Configure the Nexus deployment credentials**

Open IntelliJ's terminal and install Vim:

sudo apt install -y vim

Open the Maven settings file:

vim ~/.m2/settings.xml

Press **i** to enter Insert mode and add:

<settings>

<servers>

<server>

<id>nexus-snapshots</id>

<username>nexus</username>

<password>REPLACE_WITH_NEXUS_PASSWORD</password>

```
    </server>
  </servers>
</settings>
```

Replace REPLACE_WITH_NEXUS_PASSWORD with the password created for the nexus user in Task 2. The server ID must remain exactly nexus-snapshots because it must match the ID in pom.xml.

Press **Esc**, type :wq, and press **Enter** to save and close Vim.

Never display settings.xml in a screenshot, commit it to Git, include it in Lab6_Solution.pdf, or expose the password in terminal output.

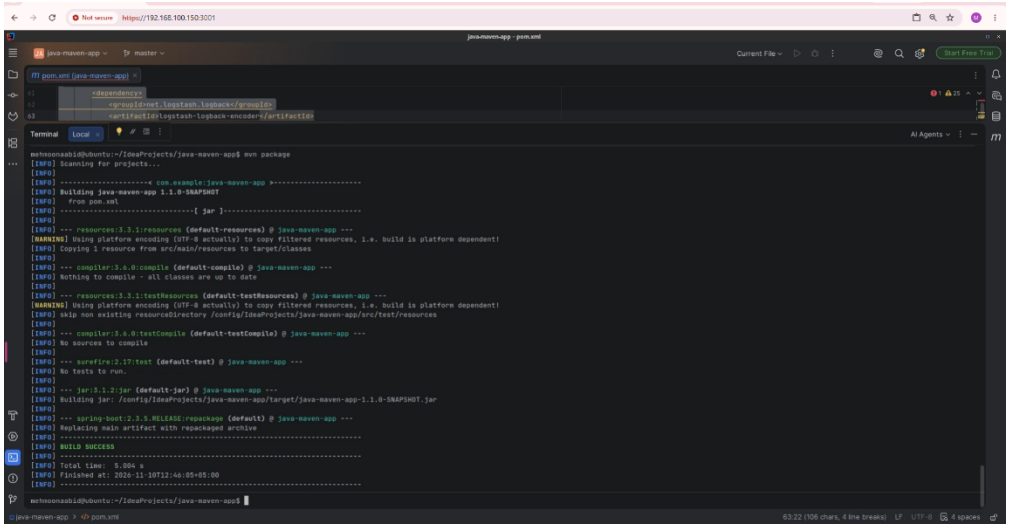
4. **Package the Maven application**

In IntelliJ's terminal, run:

```
mvn package
```

Wait for BUILD SUCCESS and verify that Maven creates the snapshot JAR under target/.

- Screenshot as maven_package_nexus.png, showing mvn package, BUILD SUCCESS, and the generated snapshot JAR filename.



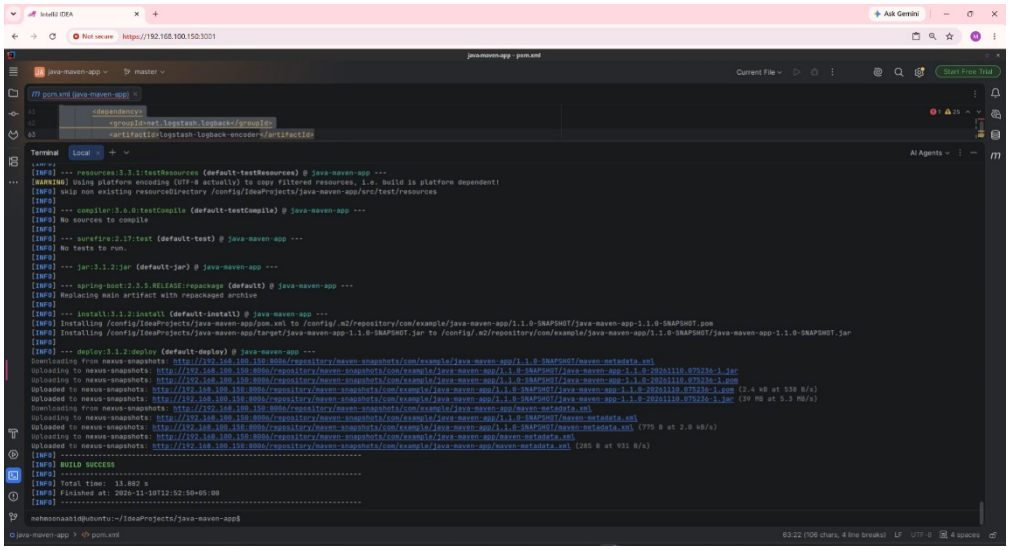
5. **Deploy the snapshot to Nexus**

Run:

mvn deploy

Maven runs the lifecycle through the deploy phase and uploads the JAR, POM, checksums and metadata to the configured Nexus snapshot repository.

- Screenshot as maven_deploy_nexus.png, showing mvn deploy, the Nexus upload lines and BUILD SUCCESS. Do not show the password or settings.xml.



If deployment returns 401 Unauthorized, verify the username, password and matching nexus-snapshots IDs. If it returns 403 Forbidden, confirm that the nexus user is Active and has the nx-java role containing nx-repository-view-maven2-maven-snapshots-*.

6. **Verify the Maven artifact in Nexus**

Open and sign in to:

http://<ubuntu-server-ip>:8006

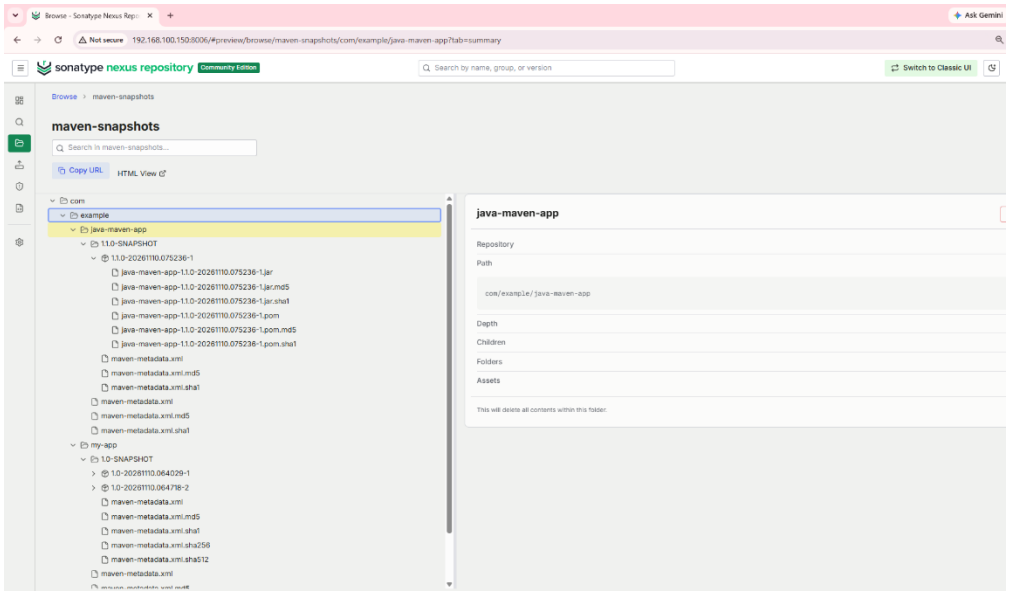
Select **Browse → maven-snapshots**, then expand:

com → example → java-maven-app → 1.1.0-SNAPSHOT

The instructor reference image shows the Maven repository hierarchy and metadata. For your required evidence, expand 1.1.0-SNAPSHOT so the published JAR, POM and metadata/checksum

files are visible. Nexus may use timestamped filenames for the snapshot.

- Screenshot as nexus_maven_artifact.png, showing the expanded com/example/java-maven-app/1.1.0-SNAPSHOT path, the JAR, POM and Maven metadata. Keep the required terminal prompt visible beside the browser.



Task 5: Inspect Nexus Repositories and Components Using the REST API

Run every command in this task directly from the **Ubuntu server terminal**. Keep the Nexus container running. These requests use the Nexus Repository REST API and compare the access available to the restricted nexus user with the admin user.

Password safety: Do not place a real password directly in the command. Using curl -u nexus or curl -u admin makes curl prompt for the password securely; the entered password is not displayed or stored as part of the command.

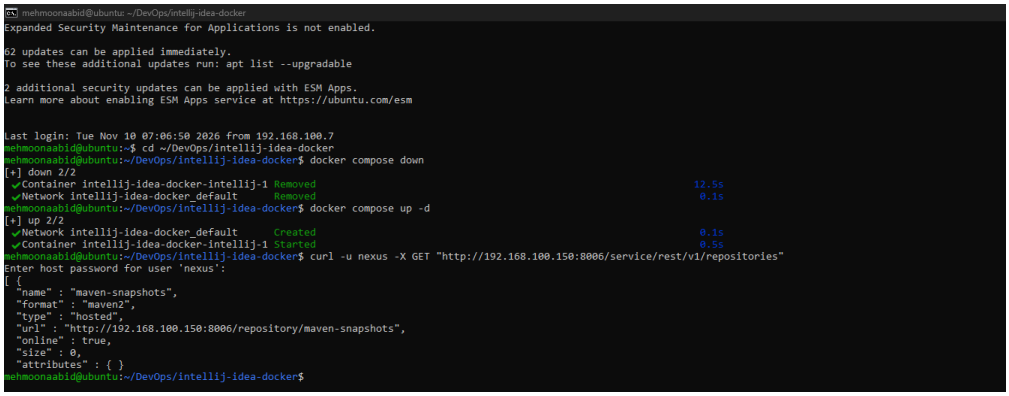
1. **List repositories as the nexus user**

Run:

curl -u nexus -X GET "http://<ubuntu-server-ip>:8006/service/rest/v1/repositories"

Enter the nexus user's password when curl prompts for it. Review the returned JSON and note which repositories are visible to this restricted user.

- Screenshot as nexus_repositories_user.png, showing the command, the repository response and your required terminal prompt. The password must not be visible.



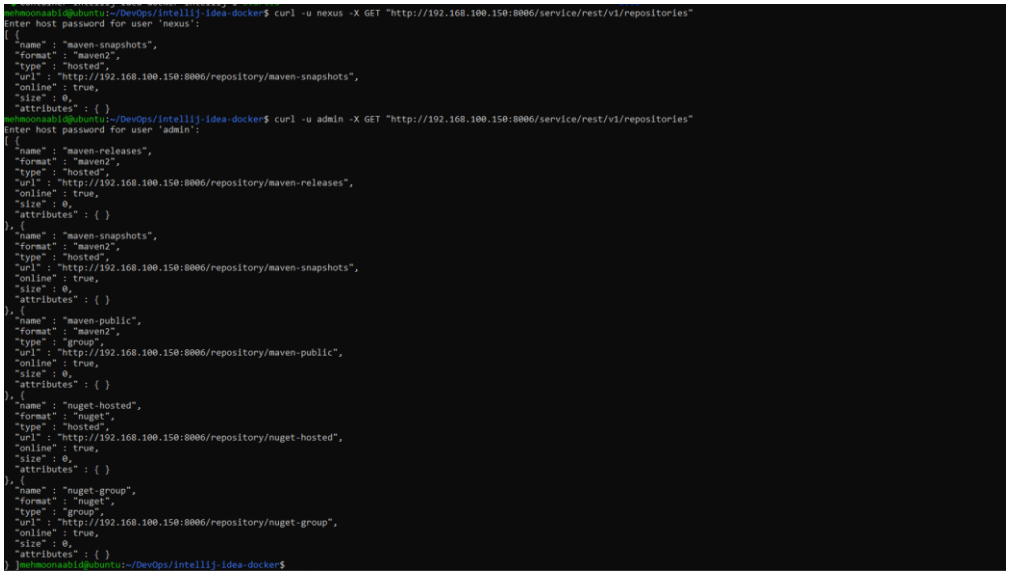
2. **List repositories as the admin user**

Run the same request with the administrator account:

curl -u admin -X GET "http://<ubuntu-server-ip>:8006/service/rest/v1/repositories"

Enter the current admin password when prompted. Compare this response with the response returned for the nexus user.

- Screenshot as nexus_repositories_admin.png, showing the command, the repository response and your required terminal prompt. The password must not be visible.



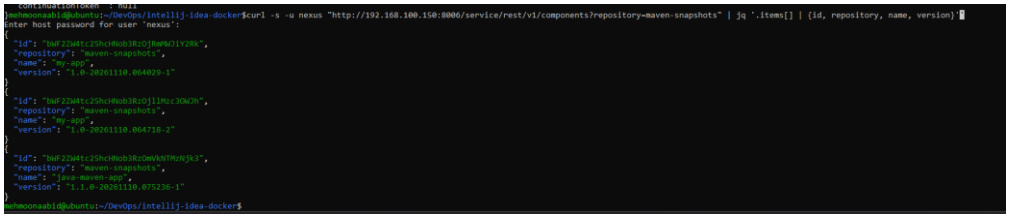
3. **List the components in maven-snapshots**

Authenticate as the nexus user and request the components stored in the Maven snapshots repository:

curl -u nexus -X GET "http://<ubuntu-server-ip>:8006/service/rest/v1/components?repository=maven-snapshots"

Enter the nexus user's password when prompted. In the JSON response, locate the items array and identify one component's id. The response can also contain a continuationToken when more results are available.

- Screenshot as nexus_components_list.png, showing the command and a returned component with its id, repository, name and version. The password must not be visible.



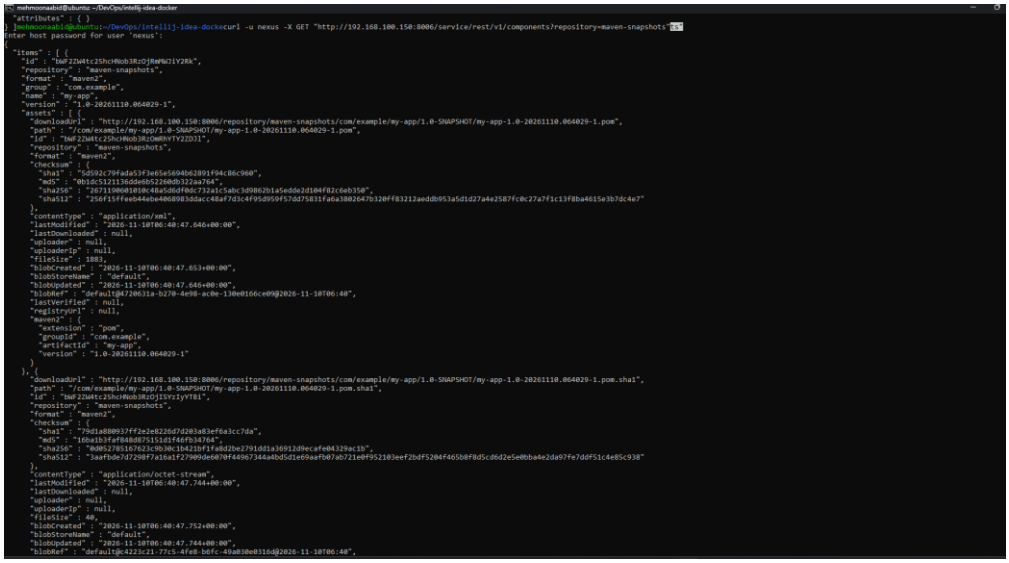
4. **Get one component's details by ID**

Copy one exact component id from Step 3 and replace <component-id> below:

curl -u nexus -X GET "http://<ubuntu-server-ip>:8006/service/rest/v1/components/<component-id>"

Enter the nexus user's password when prompted. Verify that the response describes the selected component and includes its repository, format, group, name, version and assets.

- Screenshot as nexus_component_details.png, showing the command with the selected component ID and the returned component details. The password must not be visible.



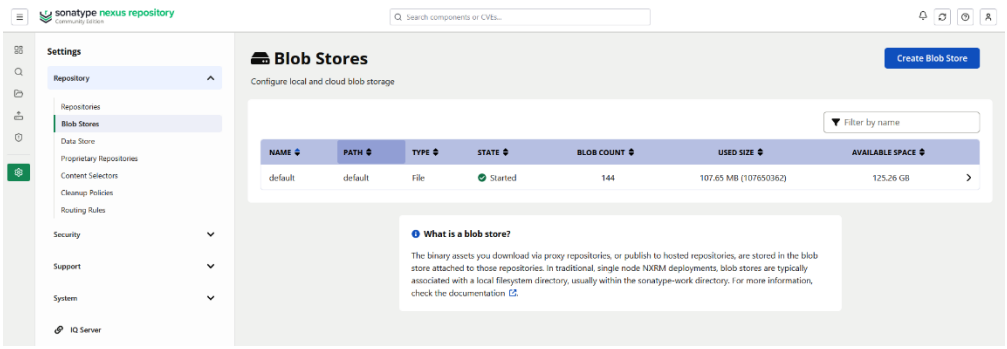
Task 6: Create a Nexus Blob Store and Docker Hosted Repository

Run the Docker commands in this task directly from the **Ubuntu server terminal**. Use the Nexus web application as the admin user for the blob-store and repository configuration.

1. **Open the Blob Stores page**

Sign in to Nexus as admin, then open:

Settings → Repository → Blob Stores



Confirm that the existing default file blob store is in the **Started** state.

2. **Inspect the default blob-store directory inside the Nexus container**

From the Ubuntu server terminal, open a shell in the Nexus container:

docker exec -it nexus /bin/sh

The container shell opens under /opt/sonatype. Run:

pwd

cd sonatype-work/nexus3

cd blobs

ls

ls default/content

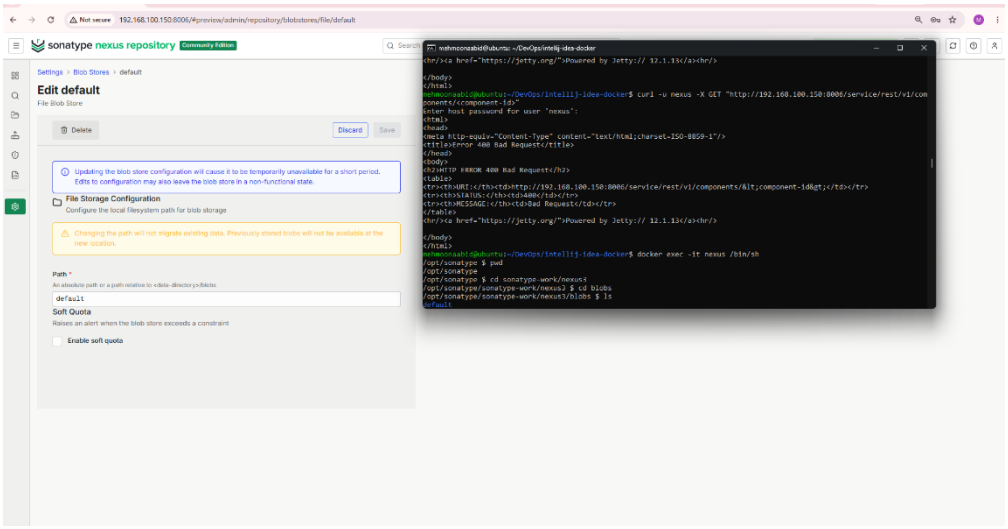
The first ls should show the default blob-store directory. The second command displays its content structure.

This is a **read-only inspection**. Do not edit, rename or delete any file inside the blob-store directory.

Keep this container shell open because it will be reused in Step 4.

- Screenshot as nexus_default_blob_content.png, showing the Ubuntu prompt that

launched docker exec, the blobs path, the default directory and the output of ls default/content.



3. **Create the mystore file blob store**

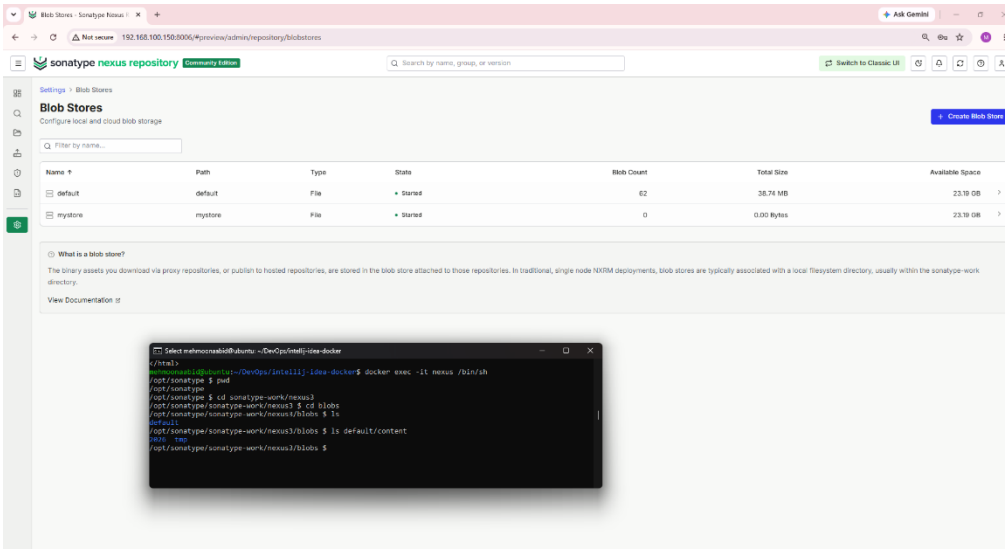
On the **Blob Stores** page, select **Create Blob Store** and configure:

- **Type:** File
- **Name:** mystore
- **Path:** /nexus-data/blobs/mystore
- **Soft Quota:** Disabled

Select **Save**. The Blob Stores page must now show both default and mystore in the **Started** state.

- Screenshot as nexus_mystore_created.png, showing both default and mystore, their File type and their **Started** state. Keep the required

Ubuntu terminal identity visible beside the browser.



4. **Verify the new blob-store directory inside the existing container shell**

Return to the Nexus container shell that you kept open in Step 2. Do not run docker exec again. Run:

pwd

ls

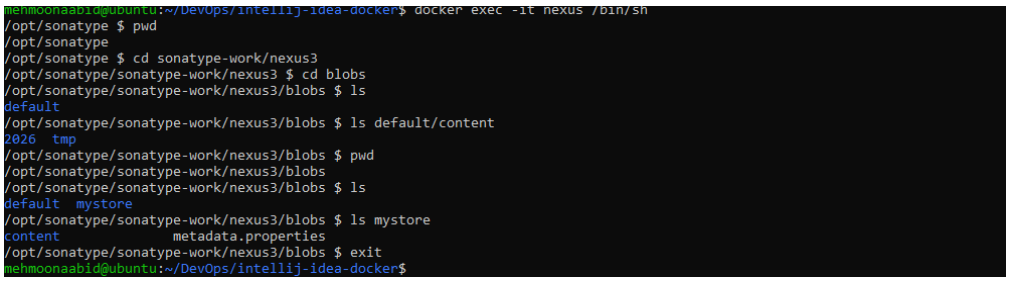
ls mystore

exit

The pwd output must show the blobs directory, and the ls output must show both default and mystore. The final exit returns you to the Ubuntu server terminal.

- Screenshot as nexus_blob_directories.png, showing the

existing container shell, the blobs path, the default and mystore directories, and the Ubuntu prompt after exit.



5. **Create a Docker hosted repository that uses mystore**

In Nexus, open:

Settings → Repository → Repositories

Select **Create repository**, choose **docker (hosted)** and configure:

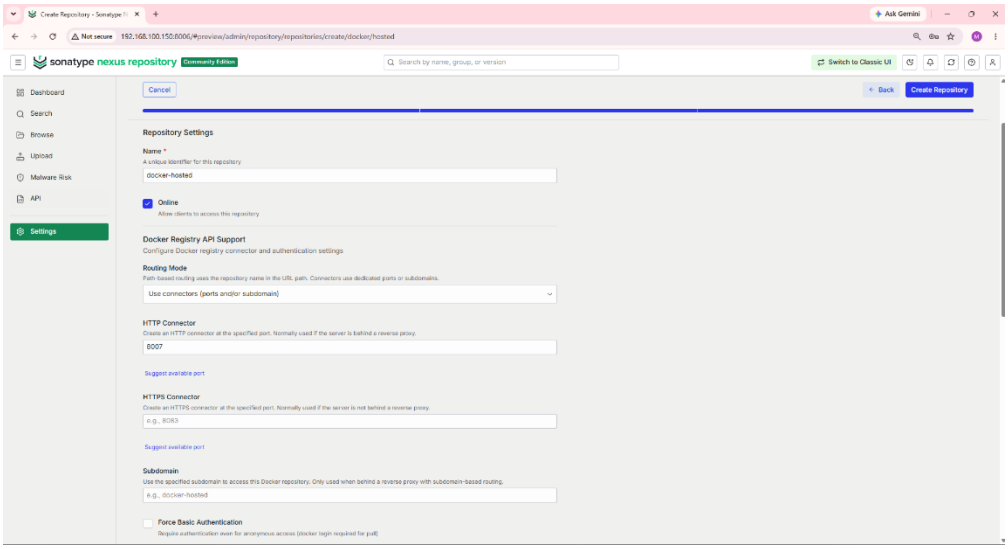
- **Name:** docker-hosted
- **Online:** Enabled
- **Repository connector: Other Connectors**
- **HTTP connector:** 8007
- **HTTPS connector:** Leave empty
- **Allow anonymous Docker pulls:** Disabled
- **Docker V1 API:** Disabled
- **Blob store:** mystore

- **Strict Content Type Validation:** Enabled
- **Deployment policy:** Allow redeploy

Use connector port 8007 because Task 1 publishes 8007:8007 for the Nexus container.

Select **Create repository**. Return to the repository list and verify that docker-hosted is **Online**, its format is docker, its type is hosted, and its blob store is mystore.

- Screenshot as nexus_docker_hosted_repository.png, showing the saved docker-hosted repository and its mystore blob-store assignment. Also show the configured HTTP connector 8007, either by reopening the saved repository or in the same readable evidence view. Keep the required Ubuntu terminal identity visible beside the browser.



Task 7: Create and Run a Maven Snapshot Cleanup Policy

This task removes every component currently visible in the maven-snapshots repository. Complete Tasks 3–5 first and capture all required publishing and REST API evidence before running this cleanup.

Destructive action: The Asset Name Matcher .* matches all snapshot assets. After the cleanup service runs, the published Gradle and Maven components will no longer appear in maven-snapshots. Do not run the task until your earlier screenshots and Lab6_Solution.pdf evidence are saved.

1. **Create the Maven cleanup policy**

Settings → Repository → Cleanup Policies → Create Cleanup Policy

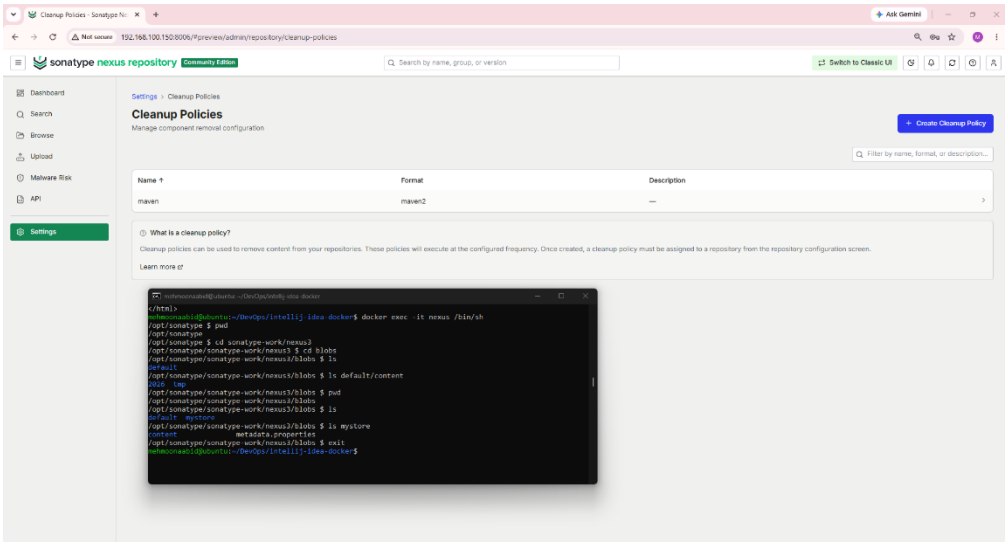
Configure:

- **Name:** maven
- **Format:** maven2
- **Release Type:** Pre-Releases/Snapshots
- **Component Age:** Not selected
- **Component Usage:** Not selected
- **Asset Name Matcher:** Selected
- **Regular expression:** .*

Under **Preview Repository**, select maven-snapshots and select **Preview**. Confirm that the preview lists the Gradle and Maven snapshot components created earlier.

Carefully review the preview, then select **Save**.

- Screenshot as nexus_cleanup_policy_created.png, showing the maven policy, maven2 format, snapshot release type, .* matcher and the populated maven-snapshots preview. Keep the required Ubuntu terminal identity visible beside the browser.



2. **Attach the policy to maven-snapshots**

Open:

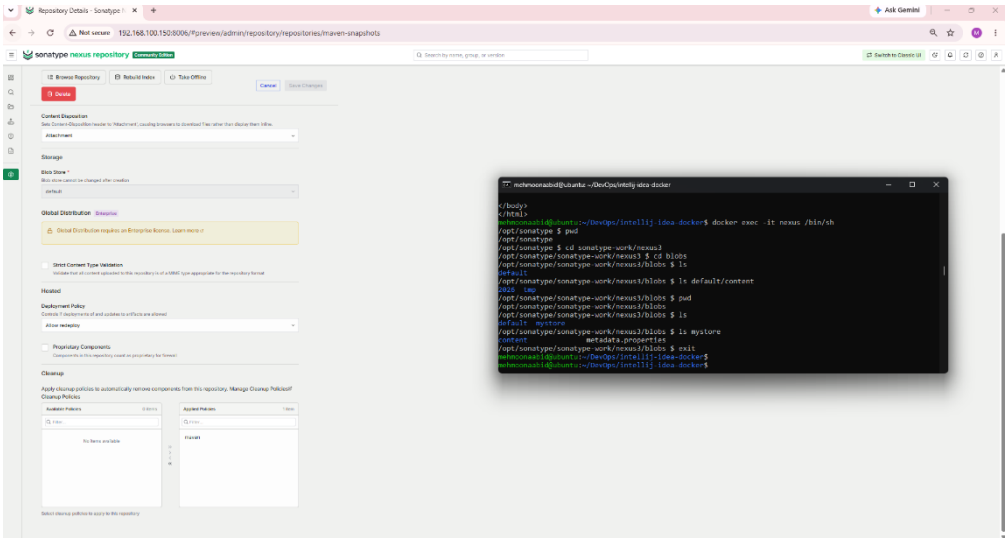
Settings → Repository → Repositories → maven-snapshots

In **Cleanup Policies**, move maven from **Available** to **Applied**, then select **Save**.

Reopen maven-snapshots and verify that maven remains in the **Applied** list.

- Screenshot as nexus_cleanup_policy_attached.png, showing the maven-snapshots repository name and the saved maven policy in **Applied**. Keep the required Ubuntu terminal identity visible beside the

browser.



3. **Verify the cleanup service schedule**

Open:

Settings → System → Tasks

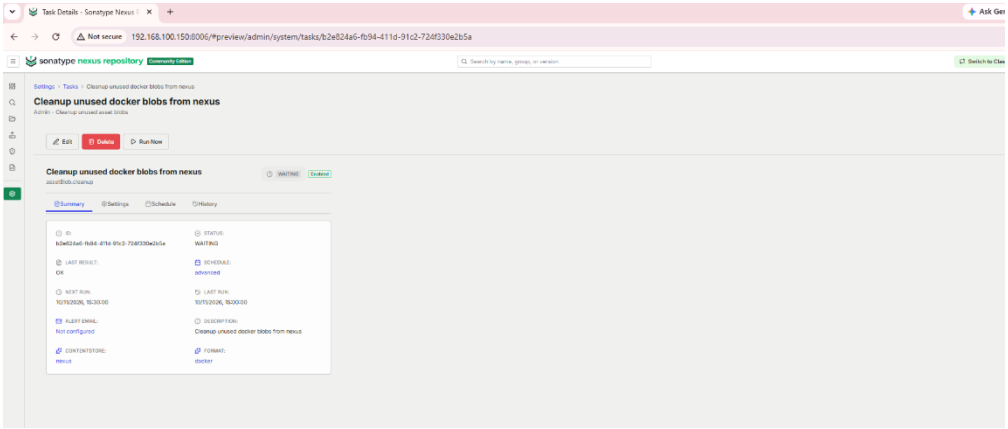
Locate **Cleanup service**, whose type is **Admin - Cleanup repositories using their associated policies**.

Open **Cleanup service** and verify:

- **Status:** Waiting
- **Next run:** The following day at 06:00:00
- **Last result:** Ok, when a previous run exists

If your instance does not show the next run at 6:00 AM, open the task's **Settings** tab, configure its daily schedule for **06:00**, save it and return to **Summary**.

- Screenshot as nexus_cleanup_task_waiting.png, showing **Cleanup service**, Waiting status and the next run scheduled for 6:00 AM. Keep the required Ubuntu terminal identity visible beside the browser.



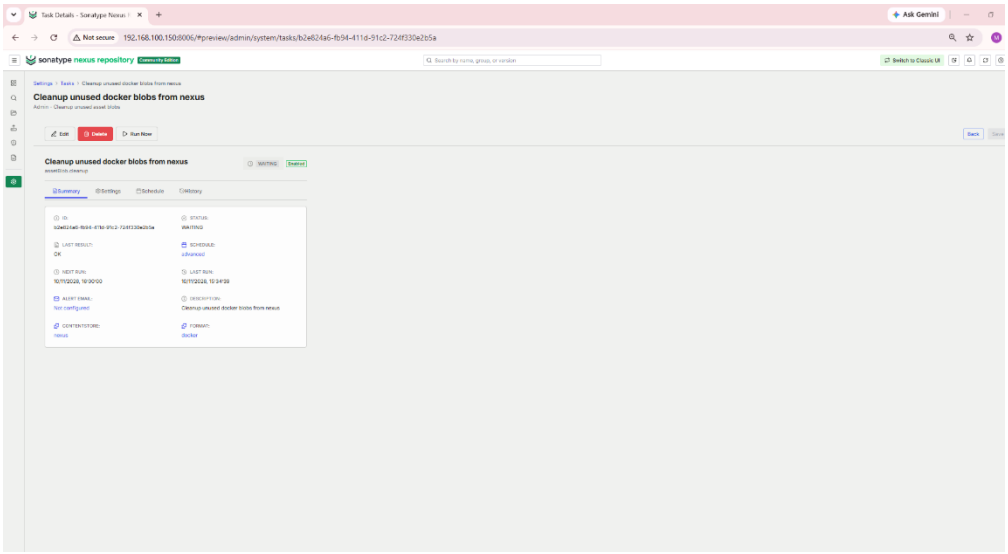
4. **Run the cleanup service manually**

Confirm once more that all earlier Lab 6 evidence has been saved. On the **Cleanup service** task page, select **Run** (the play button) and confirm the action.

Wait for execution to finish, refresh the page and verify that the task returns to **Waiting** with **Last result: Ok** and an updated **Last run** time.

- Screenshot as nexus_cleanup_task_completed.png, showing the cleanup service after the manual execution, including the updated **Last run** and **Last result: Ok**. Keep the

required Ubuntu terminal identity visible beside the browser.



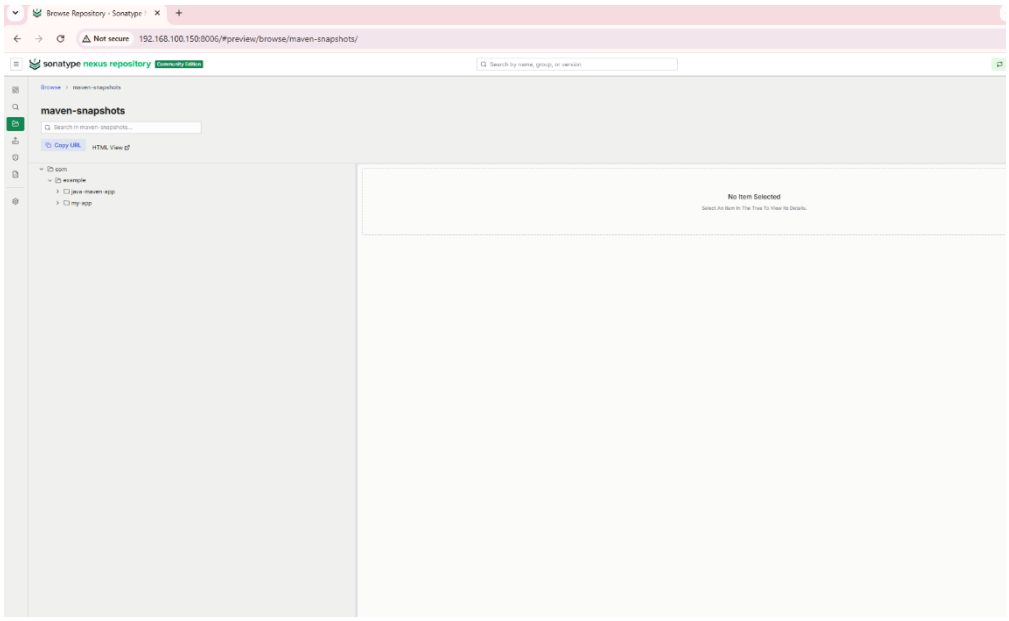
5. **Verify that maven-snapshots is empty**

Open:

Browse → maven-snapshots

The repository must display **No component/assets found in repository**.

- Screenshot as nexus_maven_snapshots_empty.png, showing the maven-snapshots repository and the **No component/assets found in repository** message. Keep the required Ubuntu terminal identity visible beside the browser.



6. **Check the blob store before compaction**

Open:

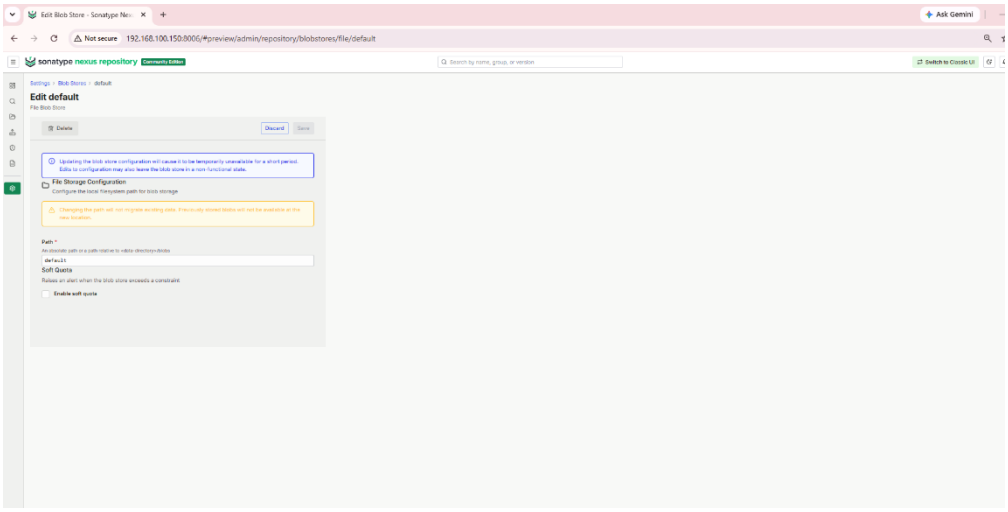
Settings → Repository → Blob Stores

The maven-snapshots repository is empty, but the default blob store can still show a high blob count and used size because the cleanup service performs soft deletion.

Record the default blob store's **Blob Count**, **Used Size** and **Available Space**. The exact values will differ between students.

- Screenshot as nexus_blob_store_before_compaction.png, showing the default blob store and its blob count, used size and available space before compaction. Keep the required

Ubuntu terminal identity visible beside the browser.



7. **Create the compact blob-store task**

Open:

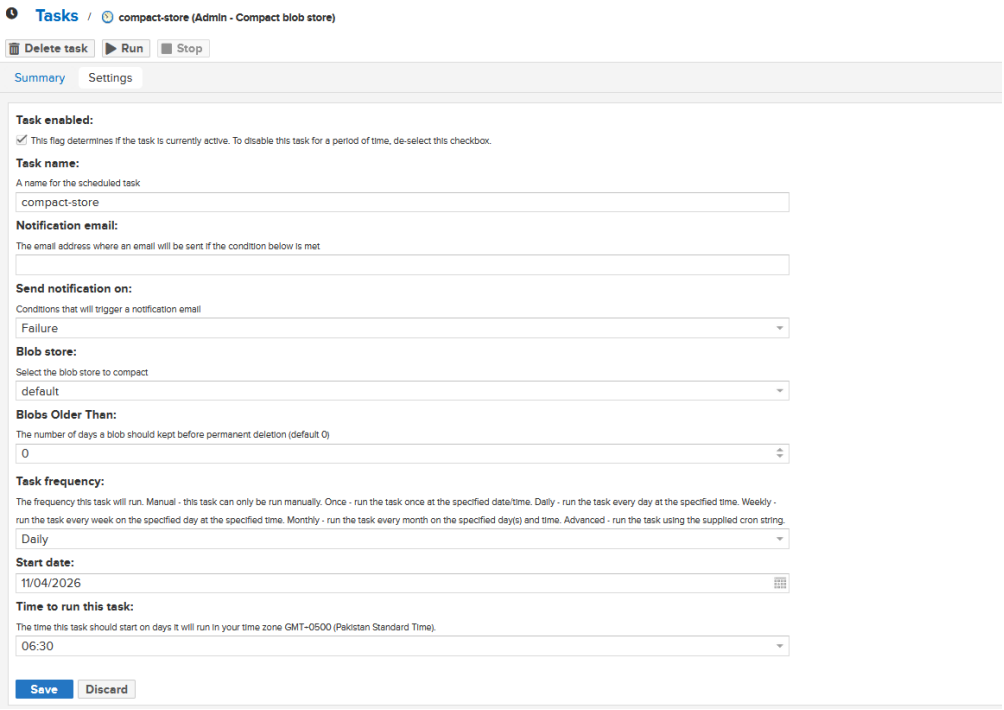
Settings → System → Tasks → Create task

In the task-type filter, enter compact, then select **Admin - Compact blob store**.

Configure the task:

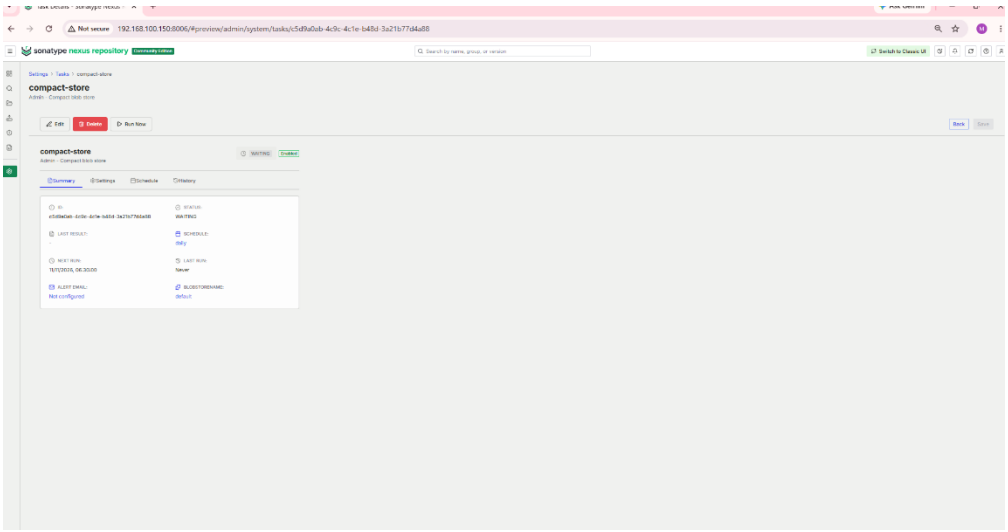
- **Task enabled:** Selected
- **Task name:** compact-store
- **Notification email:** Leave empty
- **Send notification on:** Failure
- **Blob store:** default
- **Blobs Older Than:** 0
- **Task frequency:** Daily

- **Start date:** The current date
- **Time to run this task:** 06:30



Select **Save**. Reopen compact-store and verify that it is enabled, targets default and has the configured schedule.

- Screenshot as nexus_compact_task_configured.png, showing the saved compact-store settings, default blob store, 0 days and the 06:30 daily schedule. Keep the required Ubuntu terminal identity visible beside the browser.



8. **Run the compact task manually**

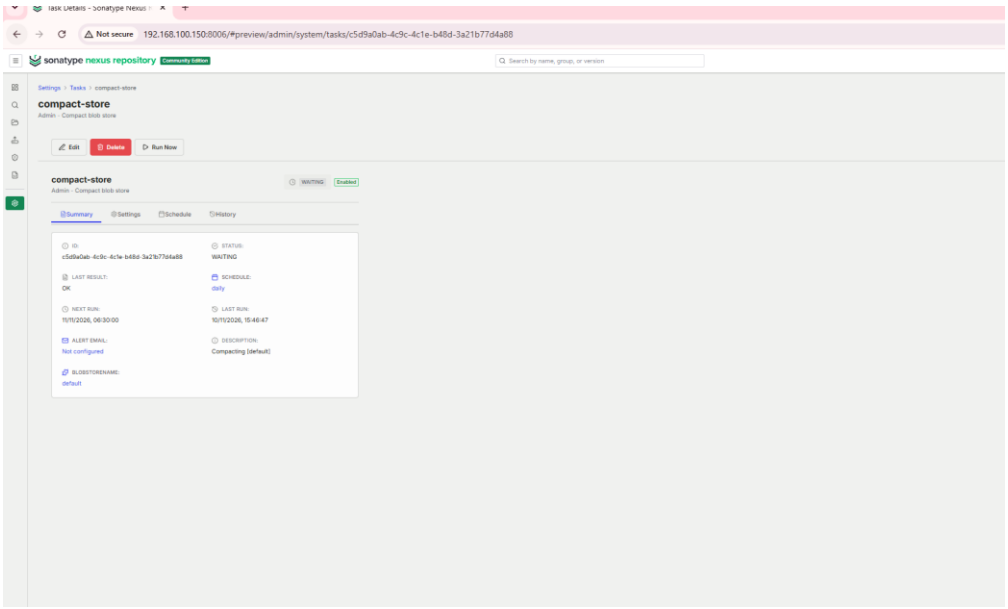
Open the compact-store task. Its Summary page should show **Type: Admin - Compact blob store** and **Status: Waiting**.

Permanent deletion: Running this task permanently removes blobs that were soft-deleted by the cleanup process and meet the 0-day setting. Confirm that every required earlier screenshot and solution-file item has already been saved.

Select **Run**, confirm the action and wait for the task to finish. Refresh until the task returns to **Waiting** and shows **Last result: Ok**.

- Screenshot as nexus_compact_task_completed.png, showing compact-store after manual execution with an updated **Last run** and **Last result: Ok**. Keep the required

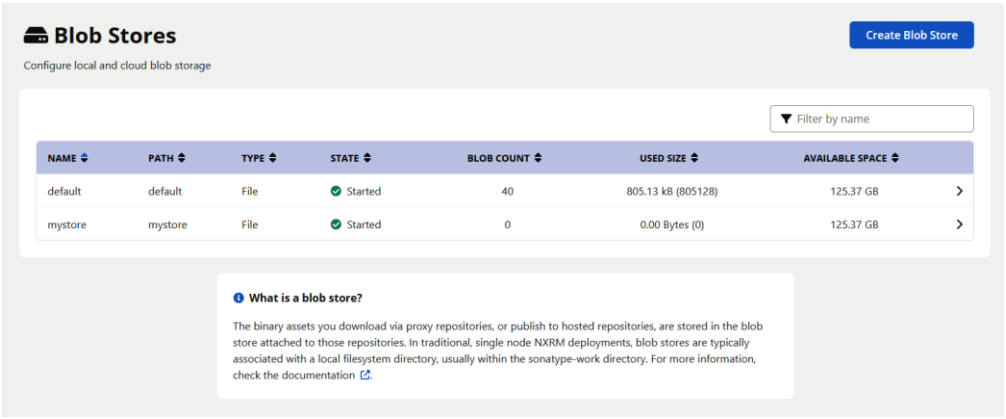
Ubuntu terminal identity visible beside the browser.



9. **Verify that blob-store space was reclaimed**

Return to:

Settings → Repository → Blob Stores



Compare the default row with Step 6. Its blob count and used size should be lower, while its available space should be higher. The exact values are not graded because they depend on the student's

previous Nexus activity. The blob store does not need to reach zero because Nexus can retain other required files.

- Screenshot as nexus_blob_store_after_compaction.png, showing the default blob store after compaction with its updated blob count, used size and available space. Keep the required Ubuntu terminal identity visible beside the browser.

